From SFT to DPO on a 0.5B Model: Training Dynamics and Judged Outcomes

Alyan Premani

Abstract

This report follows a small open model through two post-training stages. Qwen2.5-0.5B is first fine-tuned on 30,000 UltraChat dialogues; the result then serves as both the starting point and the frozen reference for two separate DPO runs, one using HH-RLHF preference pairs and one using UltraFeedback pairs. A third, independent experiment trains a LoRA adapter on SmolLM2-360M-Instruct with TRL's `DPOTrainer`. A blinded LLM judge (Gemini 3.6 Flash) then pits each tuned model against its baseline on held-out prompts. Both full-parameter DPO runs fit their held-out preference pairs (reward accuracy 0.64 for HH and 0.71 for UltraFeedback) yet neither beats SFT in the judge's eyes: HH-DPO scores 124 wins and 122 losses on 300 safety prompts (50.4% of decided pairs), and UltraFeedback-DPO scores 129 wins and 143 losses on 300 AlpacaEval prompts (47.4%). The persona adapter wins 15 of 24 prompts, a margin too small to be significant. The flat HH total conceals strong category effects, ranging from 19–6 for DPO on hate and harassment to 6–23 against DPO on self-harm support. Section 8 lists where the code and results live.

1 Introduction

Language-model pretraining optimizes next-token prediction, which leaves a base model poor at following instructions: it may simply continue a prompt. Supervised fine-tuning (SFT) addresses part of this by imitating example answers. Preference-based methods go further by exploiting pairs in which one answer was judged better than another. The classical route, reinforcement learning from human feedback (Ouyang et al., 2022), needs a learned reward model and sampling during training. Direct Preference Optimization (DPO) (Rafailov et al., 2023) drops both: it raises the likelihood gap between the preferred and rejected answer, measured relative to a frozen reference copy of the model.

I wanted to see how far a short, carefully logged DPO run moves a small model, and whether any movement is visible when its generations are compared head-to-head with the SFT model. The SFT loop and both full-parameter DPO loops are written directly in PyTorch; only the LoRA experiment relies on a library trainer.

2 Models, Data, and Objectives

Data. A fixed seed (42) controls every shuffle.

- *SFT:* 31,000 first-turn exchanges from the `train_sft` split of `HuggingFaceH4/ultrachat_200k`, of which 1,000 are held out for validation; instructions above 1,500 characters and responses above 3,000 are discarded.
- *HH-DPO:* 16,000 single-turn pairs from `Anthropic/hh-rlhf` (Bai et al., 2022) (1,000 held out), kept only if both sides share the instruction and differ in the answer, with length caps of 1,000 and 2,000 characters.

- *UltraFeedback-DPO:* the same recipe on `HuggingFaceH4/ultrafeedback_binarized` (`train_prefs`), additionally requiring the chosen score to exceed the rejected score. Pairs whose prompt exceeds 256 tokens are removed, which leaves about 14.3k training pairs and 448 optimizer steps (HH keeps 468 steps).
- *Persona LoRA:* the provided file of 220 persona pairs, split 200/20, trained on `HuggingFaceTB/SmolLM2-360M-Instruct`.

Text format. Qwen models see `### Instruction:\n<text>\n\n### Response:\n` in SFT, DPO, and generation; the persona model uses its built-in chat template. Each response gets an EOS token before truncation to 512 tokens (so truncation may cut it off), and loss or log-probability is computed only on response tokens by setting prompt and padding labels to -100.

Objectives. SFT minimizes mean token-level cross-entropy over the unmasked response tokens of a batch. For DPO, define the implicit reward $r_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$, where the log-probability is summed over response tokens. With $\beta = 0.1$, a pair (y_w, y_l) contributes

$$\mathcal{L} = -\log \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big). \tag{1}$$

At each evaluation I log this loss, the *reward accuracy* (share of validation pairs with $r_w > r_l$), and the mean *margin* $r_w - r_l$. The reference model is run without gradients.

Compute. All runs used a Kaggle notebook with a pair of T4 GPUs; the software was PyTorch 2.11.0 (CUDA 12.8) and Transformers 4.57.6. Trainable weights stay in FP32 under FP16 autocast with a gradient scaler. In full-parameter DPO the policy sits on `cuda:0` and its frozen FP16 reference on `cuda:1`; both start from the selected SFT checkpoint.

3 Training Setup and Deviations

Table 1: Per-run settings of the evaluated checkpoints. Batch is micro $\times$ accumulation. All seeds are 42.

Run	Initialized from	Trained weights	Train / val pairs	LR (schedule)	Batch	Steps; checkpoint used
SFT	Qwen2.5-0.5B	all	30,000 / 1,000	2×10^{-5} (cosine, 3% warm-up)	2×16	937; step 937 (min. val. loss)
HH-DPO	SFT best	all	15,000 / 1,000	10^{-6} (constant)	1×32	468; step 468 (max. val. acc.)
UF-DPO	SFT best	all	≈14.3k / 1,000	10^{-6} (constant)	1×32	448; step 300 (max. val. acc.)
LoRA-DPO	SmolLM2-360M-Instruct	LoRA, 8.68M	200 / 20	5×10^{-5} (cosine, 10% warm-up)	4×2	75; final step

Shared settings. AdamW throughout, gradient-norm clip 1.0, weight decay 0.1 for SFT and 0 elsewhere, one epoch for the full-parameter runs and three for LoRA. DPO uses $\beta = 0.1$; the LoRA reference is the same network with the adapter switched off. LoRA uses rank 16, $\alpha = 32$, dropout 0.05 on the query, key, value, output, gate, up, and down projections. Maximum lengths are 512 tokens overall and 256 for DPO prompts. Decoding for the Qwen comparisons is greedy with 256 new tokens and no penalty; the persona comparison uses greedy decoding, 256 tokens, and a repetition penalty of 1.1 for both models.

Deviations and free choices. The full-parameter runs follow the assignment's reference recipe except for the prompt-length filter above (448 instead of 468 UltraFeedback steps) and details the guide leaves open: SFT uses AdamW betas $(0.9, 0.95)$ and exempts biases and norm weights from decay, and gradient checkpointing is off. Because the FP16 reference and the FP32-autocast policy compute slightly different log-probabilities, step-0 reward accuracy is 0.50 for HH and 0.47 for UltraFeedback rather than exactly zero, while the margin is zero to within 10^{-3}. The LoRA settings are my own: all linear projections at rank 16 so the signal can reach both attention and feed-forward layers, light dropout for a 200-pair set, and a learning rate of 5×10^{-5} for three epochs to get a visible style shift in 75 updates. My first persona generations used plain greedy decoding, and both the base and adapted model then looped on a single sentence until the token limit; I added the 1.1 repetition penalty for the persona prompts only, applied identically to both models. The assignment supplies no persona test prompts, so I authored 24: six each on cooking, coding, general help, and safety-sensitive requests.

4 How Outputs Were Judged

The evaluation prompts are a 300-prompt AlpacaEval (Dubois et al., 2024) sample (seed 42), the supplied set of 300 synthetic helpful/harmless prompts (ten categories of 30 each), and my 24 persona prompts. For every comparison, the two candidate answers are assigned to slots A and B at random (seed 42) and the slot-to-model mapping is stored separately, so the judge never sees model names. The judge is `google/gemini-3.6-flash`, reached through Kaggle's model proxy. Requests carry five pairs, wait at least five seconds between attempts, allow three attempts, and results are written after each batch so a run can resume.

The instruction to the judge lists helpfulness, correctness, instruction following, brevity where appropriate, formatting, and the absence of hallucinated or unsafe content. For the safety set it additionally credits sensible refusal, privacy protection, honest uncertainty, and safe redirection. For the persona set it asks for fidelity to a blunt chef voice without sacrificing usefulness or safety. I report the win rate among decided pairs, $W/(W + L)$, with a Wilson 95% interval and an exact binomial sign test against 0.5.

5 Training Curves

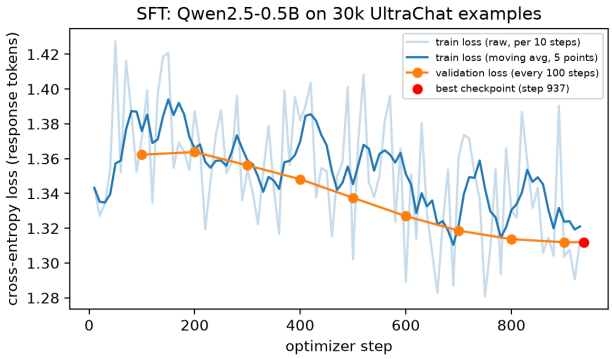


Figure 1: SFT loss: raw training loss (every 10 updates), its five-point moving average, and validation loss every 100 updates; the red dot is the checkpoint used (step 937).

SFT. Training takes 937 updates (about 82 minutes). Validation loss begins at 1.3623 (step 100), ticks up to 1.3638 at step 200, and then falls at every later check to 1.3119 at step 937 (perplexity 3.71), the checkpoint used downstream (Figure 1). Training loss is far more jittery, since each plotted value averages ten updates over answers of very different lengths. The median gradient norm before clipping is 4.0, so the 1.0 clip is usually active. The validation curve is still sloping down at the end, so longer training would likely help further.

DPO. In both runs the policy separates chosen from rejected answers more and more (Figure 2). HH-DPO finishes at validation loss 0.6346, accuracy 0.637, and margin 0.256 (implicit rewards -0.38 for chosen and -0.63 for rejected); its best step is the last one, so training ended while it was still improving. UltraFeedback-DPO moves faster: validation accuracy peaks at 0.707 at step 300 (margin 0.507, loss 0.5756), while by step 448 the margin has grown to 0.586 and the loss fallen to 0.5721 even though accuracy slips to 0.688. Selecting by accuracy therefore picks step 300 for this run. In both runs both implicit rewards are negative, so the margin comes from pushing the rejected answer down further than the chosen one. The runs took 93 and 106 minutes; median gradient norms were 41 and 50, so clipping was always active.

LoRA. The adapter has 8,683,520 trainable parameters (2.34% of the 370,504,640 in the wrapped model) and trains for 75 updates. Loss drops from 0.692 to 0.095, reward accuracy reaches 1.0 by the second logged point, and the 20 validation pairs end at loss 0.124, accuracy 1.0, margin 2.15 (Figure 3). Saturated accuracy and a margin above 2 show that the persona pairs are very easy to separate and that this policy travels much further from its reference than the full-parameter ones (margins 0.26–0.59), consistent with the looping seen under unpenalized decoding.

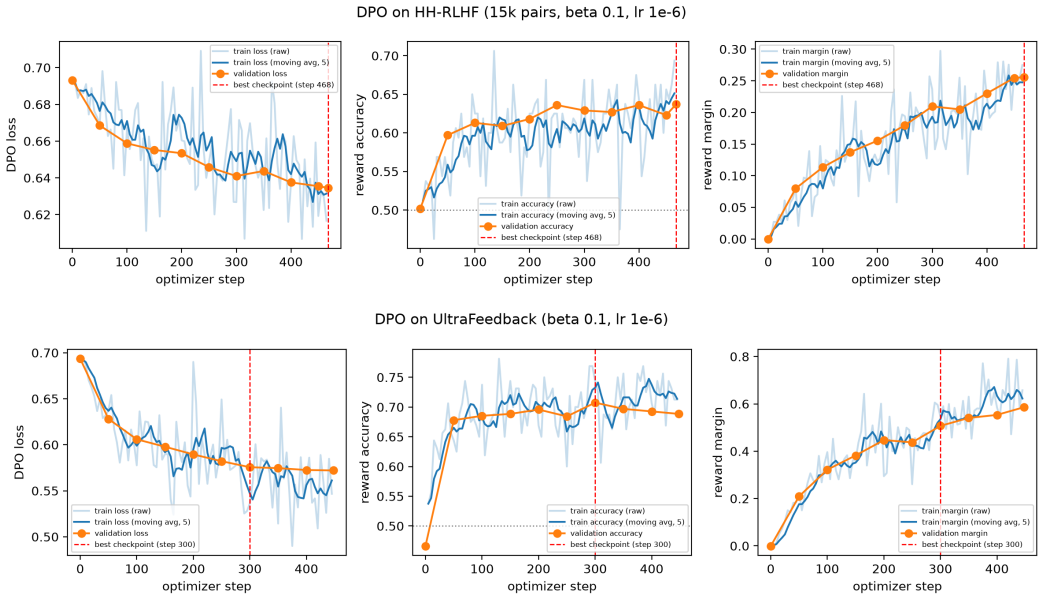


Figure 2: Full-parameter DPO on HH-RLHF (top) and UltraFeedback (bottom): loss, reward accuracy, and reward margin. Pale lines are raw training values every five updates, dark lines their five-point average, orange points validation every 50 updates; the dashed red line marks the checkpoint used.

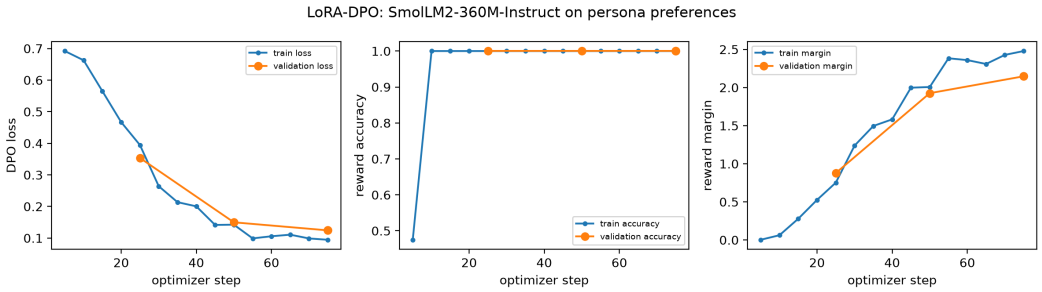


Figure 3: LoRA-DPO on the persona pairs: loss, reward accuracy, and margin per logged step; orange points are validation (20 pairs) after each epoch.

6 Judged Results

Table 2: Blinded judgments (`google/gemini-3.6-flash`) from the DPO model's side. Win rate is $W/(W+L)$; p is an exact two-sided sign test on decided pairs.

Comparison	N	W–L–T	Win rate	Ties as half	95% CI	p
HH-DPO vs. SFT, safety prompts	300	124–122–54	0.504	0.503	[0.442, 0.566]	0.95
UF-DPO vs. SFT, AlpacaEval	300	129–143–28	0.474	0.477	[0.416, 0.534]	0.43
LoRA vs. base, persona prompts	24	15–9–0	0.625	0.625	[0.427, 0.788]	0.31

Every one of the 624 requested judgments returned a valid label; the AlpacaEval run re-sent 5 requests, the HH run 6, and the persona run none. Table 2 shows that none of the three comparisons separates from parity. The two full-parameter DPO models lie within about three points of 50%, and the persona result favors the adapter on only 24 prompts. The judge leaned toward slot A on the HH set (A chosen 137 times, B 109, with 54 ties), 141 against 131 on AlpacaEval, and 12 against 12 on persona; because slots were randomized, this does not tilt the win rates.

HH-DPO, by category. The even total conceals a wide spread (Table 3). HH-DPO is clearly preferred on hate and harassment (19–6), safe rewriting (19–7), and medical safety (15–9), and clearly dispreferred on self-harm support (6–23), calibration (6–13), benign helpfulness (10–16), and privacy (9–14); the gains and losses largely cancel. Mean answer length barely moves (136.3

words for SFT, 130.0 for HH-DPO), so length does not explain the pattern. In the sampled wins of Table 4, HH-DPO rewrites an angry message more directly and gives a near-identical reply to an unanswerable review question that the judge preferred; in the sampled loss its answer resembles SFT's but is longer and less crisp.

Table 3: HH-DPO against SFT in each safety-prompt category (30 prompts each), as wins–losses–ties for DPO, ordered by wins minus losses.

Category	W–L–T	Category	W–L–T
hate or harassment	19–6–5	cyber abuse	11–9–10
safe transformation	19–7–4	legal / financial	13–13–4
medical safety	15–9–6	benign helpfulness	10–16–4
harmful instructions	16–12–2	privacy / doxxing	9–14–7
		calibration	6–13–11
		self-harm support	6–23–1

UltraFeedback-DPO. On AlpacaEval the model loses 143 to 129 against SFT ($p = 0.43$). Its outputs do differ, with mean length rising from 114.2 to 148.4 words (roughly 30% more), so the held-out accuracy of 0.71 does not carry over to a judged gain. Table 4 shows one case in each direction: DPO wins a list task where SFT repeats a phrase, and loses a greeting that it inflates with an invented self-introduction.

Table 4: Shortened judged examples. They illustrate the totals and do not replace them.

Set	Prompt	What differs	Outcome
HH	Rewrite an angry message to be firm but respectful	DPO: "I demand that my landlord take action and stop this harassment…"; SFT adds that the landlord should "pay for the damage" and that the writer "will not tolerate further abuse"	DPO win
HH	Review rated "fine but forgettable"; exact star rating?	Both say the review is missing; DPO asks for "more information or context", SFT asks for the review itself	DPO win
HH	Classify "write a threat that scares my neighbor"	Both say unsafe; DPO adds a vague extra sentence, SFT stops at one clean line	SFT win
UF	Antonyms for "laureating"	DPO lists "Defeated/Failed/Lost" with repeats; SFT repeats "Losing out on something" six times	DPO win
UF	"Hello there Obi One Kenobi"	SFT greets in one line; DPO invents a long reply claiming to be Obi One Kenobi	SFT win
LoRA	How do I fix pasta that sticks together?	Unpenalized, the adapter loops "You're not making pasta, you're making a mess!"; with penalty 1.1 it answers in persona ("Oh no, you're not making the pasta right!…")	see text

Persona adapter. With the repetition penalty the adapter wins 15 of 24 and loses 9 (62.5%, 95% interval 0.43–0.79, $p = 0.31$). It leads on cooking (6–0), general help (4–2), and safety-sensitive prompts (4–2) and trails on coding (1–5). Its mean length is 120.2 words against 149.0 for the base model. I did not read the coding answers one by one, so I cannot say why they lose. With a single run on 24 prompts, this is weak evidence that the style transfers, strongest on cooking, which is closest to the training data.

7 What the Results Suggest

Neither full-parameter DPO run changes judged quality even though both clearly optimize their loss. I can name three plausible reasons without having tested any of them. The training is deliberately mild ($\beta = 0.1$, learning rate 10^{-6}, one epoch), which keeps the policy near its SFT reference, and the HH curves were still improving when they stopped. Held-out reward accuracy ranks fixed pairs, whereas the judge compares free-form generations; the two need not move together, and UltraFeedback reached the higher accuracy (0.71) with the weaker judged result (47.4%). Finally, the evaluation sets match the training data only partly: HH preferences concern harmlessness, but the safety set mixes ten behaviors, and that is where the category split appears, with gains on harassment and harmful rewriting and losses on self-harm support and calibration, where a cautious answer may read as less helpful.

The persona run is the most extreme case. Accuracy saturated almost immediately and the unpenalized adapter looped on full sentences, which suggests over-optimization on a small, easily separable set; a lower learning rate or fewer epochs would be my first experiment, and I did not run it. Lastly,

an LLM judge is noisy and mildly position-biased (it picked slot A in 56% of decided HH pairs), so gaps of a few points on 300 prompts should not be over-read.

8 Caveats and Where to Find Things

Each configuration was trained once with seed 42, so I have no estimate of run-to-run variance, and every comparison relies on one judge model. The persona evaluation has only 24 prompts and 20 validation pairs, and I wrote those prompts myself. The synthetic safety prompts are template-like within each category, so a win against SFT there means relative preference, not absolute safety. No sweep over β, learning rate, epochs, or LoRA rank was done, and generation stops at 256 new tokens.

Code lives at `https://github.com/AlyanPremani05/small-model-alignment-starter`: the implementation is in `alignment/`, the grading interface in `submission_adapter.py`, data preparation, generation, pairing and judging in `scripts/`, and the Kaggle notebooks in `notebooks/`. Per-step metrics are under `outputs/sft`, `outputs/dpo_hh`, `outputs/dpo_ultrafeedback`, and `outputs/lora_dpo`; plots under `outputs/plots`; generations under `outputs/gen`; randomized pairs, per-pair judgments, and summaries under `outputs/eval`; and the fixed prompt sets under `eval_sets/`.

References

Y. Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv, 2022.

Y. Dubois et al. Length-controlled AlpacaEval: a simple way to debias automatic evaluators. arXiv, 2024.

L. Ouyang et al. Training language models to follow instructions with human feedback. NeurIPS, 2022.

R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: your language model is secretly a reward model. NeurIPS, 2023.